

Overview

bUSD: Bitcoin's Native Stablecoin

Breaking new ground as Bitcoin's first native stablecoin—where stability meets sovereignty.

bUSD is Bitcoin's own stable asset—born from its native infrastructure, secured by its collateral, and represented as Runes.

No bridges. No custodians. No compromises. Just pure permissionless finance, fully aligned with Bitcoin's principles of decentralization and censorship-resistance.

Every operation—minting, redemption, liquidation—executes atomically, ensuring consistency, reliability, and uncompromising stability.

And now, with Internet Computer (ICP) as its execution layer, these guarantees extend beyond Bitcoin's base layer.

ICP brings decentralized computation, native Bitcoin smart-contract capabilities, and threshold-signed taproot transactions directly on-chain—without wrapping or intermediaries.

The stablecoin canister uses ICP's Bitcoin API, Schnorr threshold signatures, and tamper-proof state replication to enforce the rules of bUSD with mathematical finality.

ICP does not replace Bitcoin—it complements it:

- Bitcoin provides the collateral and settlement.
- ICP provides computation, automation, vault management, and MPC key security.
- Together they form a sovereign, trust-minimized financial stack rooted directly in Bitcoin.

With bUSD, Bitcoin becomes more than a store of value—it becomes the foundation for lending, saving, payments, and trading in a trust-minimized ecosystem.

Step into the era of true monetary sovereignty—powered jointly by Bitcoin and ICP.

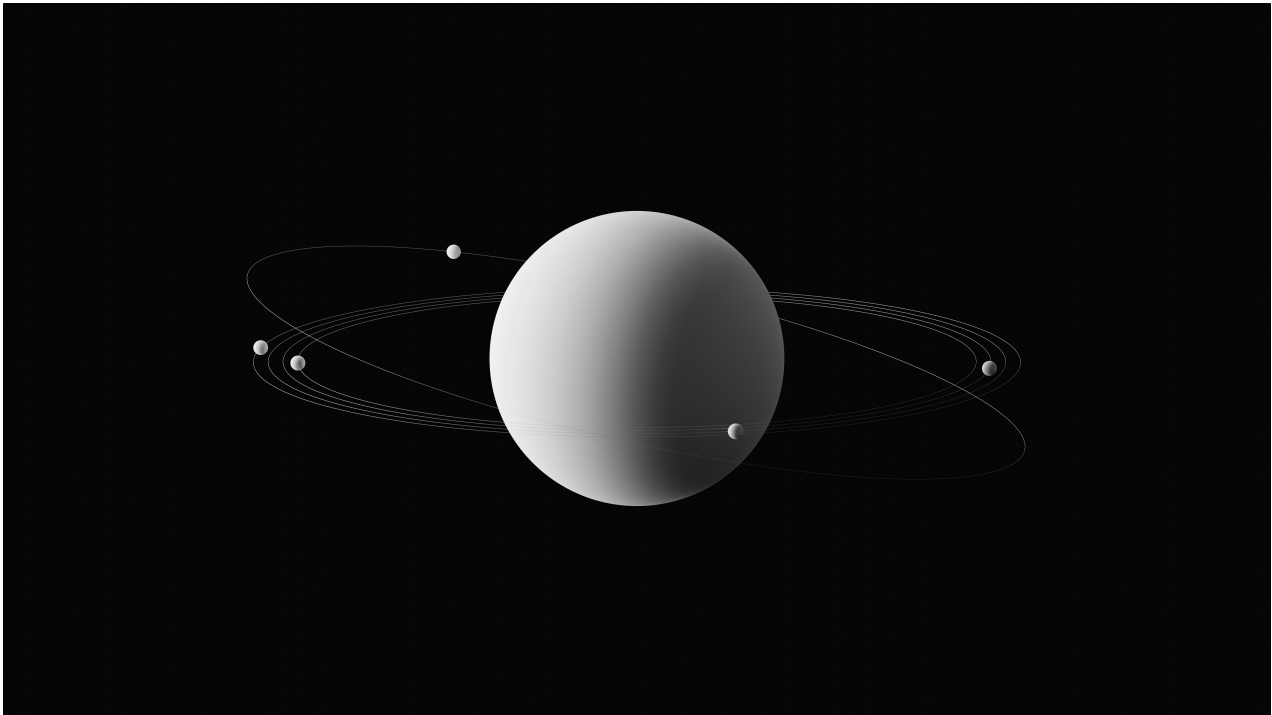
BTC Stablecoin on ICP

2e36g-4qaaa-aaaae-acxka-cai.icp0.io



Disclaimer : We use the rune USDBZ.STABLECOIN to represent bUSD which is deployed on Bitcoin testnet4 for this project.

Jump right in



Atomicity

Create your first site



Sovereignty

Learn the basics of GitBook

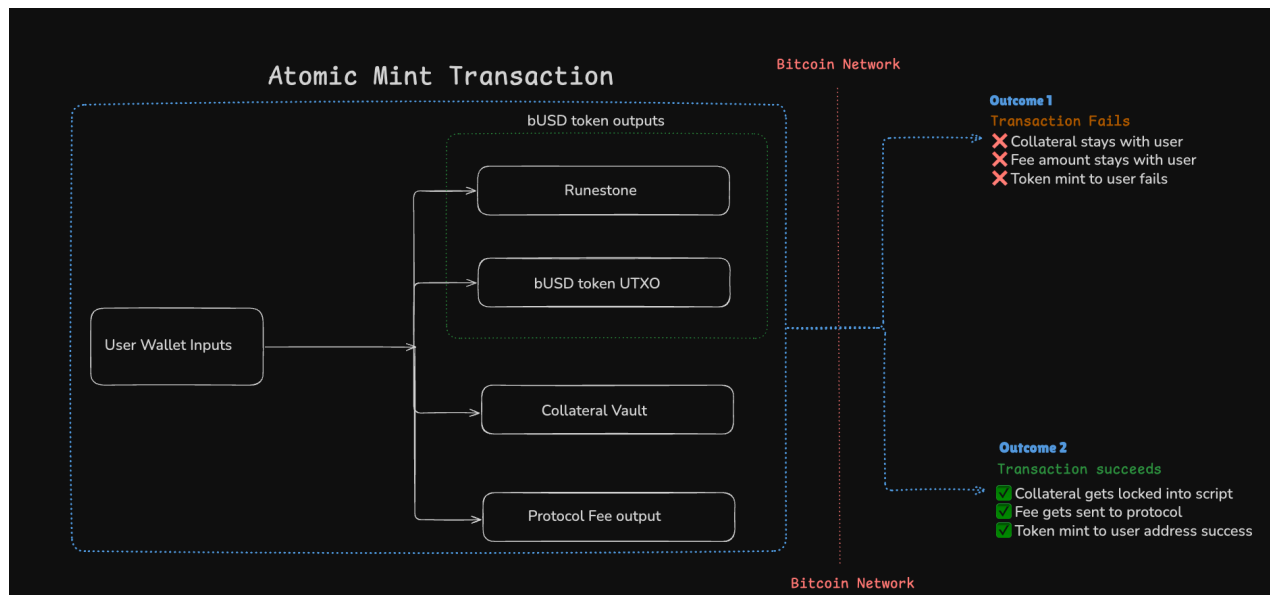


Stability

Share your docs online

Atomicity

Nothing partial. Nothing broken. Only atomic.



Atomicity is a critical property of the bUSD protocol. It ensures that each core operation—minting, withdrawing, and claiming—executes entirely or not at all, preventing partial state changes and preserving system integrity.

Why Atomicity matters

Consistency in Distributed Systems

Atomic operations guarantee that complex, multi-step processes complete without intermediate, inconsistent states. In USDB, collateral locking and token issuance are combined into a single transaction, ensuring either both actions succeed or neither occurs.

Limitations of Multi-Step Approaches

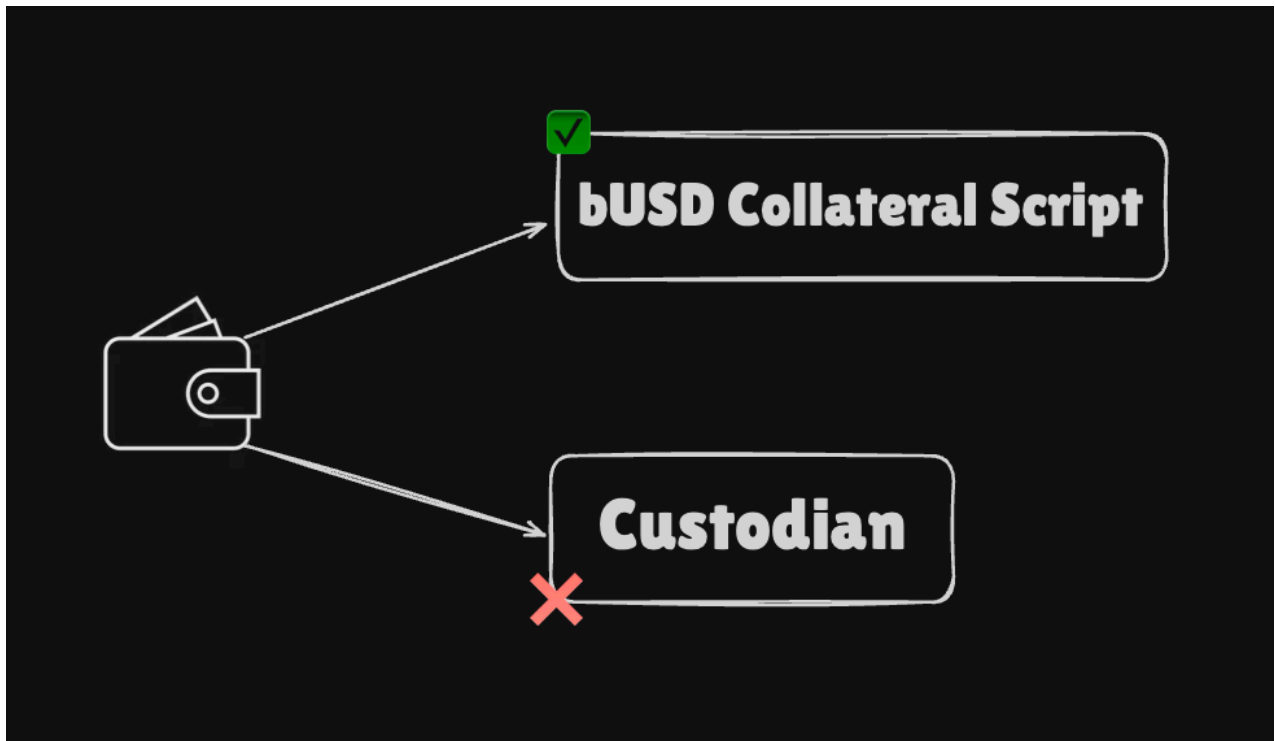
Protocols that split operations across multiple transactions introduce risk: failed intermediate steps can result in locked collateral without token issuance, or tokens burned without collateral release. Such failures undermine user confidence and require manual intervention which contradicts the notion of Decentralisation.

Benefits for Users

By executing all steps in a single transaction, USDB ensures users never face partial failures—collateral is either locked and tokens are issued together or not at all. This atomic approach delivers predictable outcomes: every action yields a definitive result, eliminating uncertainty and minimising mistakes.

Sovereignty

Your Bitcoin. Your control. Your sovereignty.



Self-Custody by Design

bUSD ensures users maintain primary control over their Bitcoin collateral through non-custodial, Taproot-based script paths. Every interaction—minting, redeeming, and claiming—executes directly on the Bitcoin blockchain, enforced by deterministic spending conditions and threshold-secured signatures. With ICP's native Bitcoin integration, these conditions are validated and signed using decentralized threshold Schnorr keys rather than trusted operators.

Why Sovereignty Matters

Preserving User Control

Users retain full ownership of their Bitcoin throughout the collateral lifecycle.

Collateral is locked into transparent Taproot scripts that define the protocol rules while maintaining the user as the primary controller.

All standard operations—mint, withdraw—require the user's signature, protecting them from custodial capture.

ICP's threshold Schnorr key derivation is used *only* to co-sign protocol-restricted paths (such as liquidation or recovery), without ever taking custody of user collateral.

This eliminates counterparty risk while enabling advanced on-chain logic.

Eliminating Single Points of Failure

Traditional stablecoins require users to trust centralized issuers who hold reserves and can freeze funds at will.

bUSD removes these trust bottlenecks by using:

- Bitcoin-native script conditions for vault creation and collateral management
- ICP's decentralized threshold signing for protocol signatures
- No hot wallets, no custodial keys, no centralized servers controlling funds

All signatures from the protocol side are generated by replicated threshold MPC nodes on the Internet Computer, where the private key never exists on any machine.

This ensures no single participant—including the protocol authors—can unilaterally move or block funds.

Maintaining Censorship Resistance

With no centralized custodian, operator, or backend holding private keys, bUSD inherits Bitcoin's censorship resistance directly.

Users interact:

- Directly with Bitcoin UTXOs
- Directly with deterministic Taproot script paths

- With protocol signatures produced only through threshold consensus on ICP

No single entity can reverse transactions, freeze vaults, or censor redemption flows.

The Internet Computer acts only as a decentralized signer—never a custodian.

How Sovereignty is Enforced

bUSD uses multi-path Taproot scripts where each branch corresponds to a transparent, auditable rule:

- User-controlled spend path

For normal withdrawals, users sign with their own payment key.

- Liquidator/Oracle liquidations (threshold Schnorr)

If collateral falls below required ratio, ICP's threshold-signed leaf activates liquidation logic—but only with oracle attestation and deterministic conditions enforced on-chain.

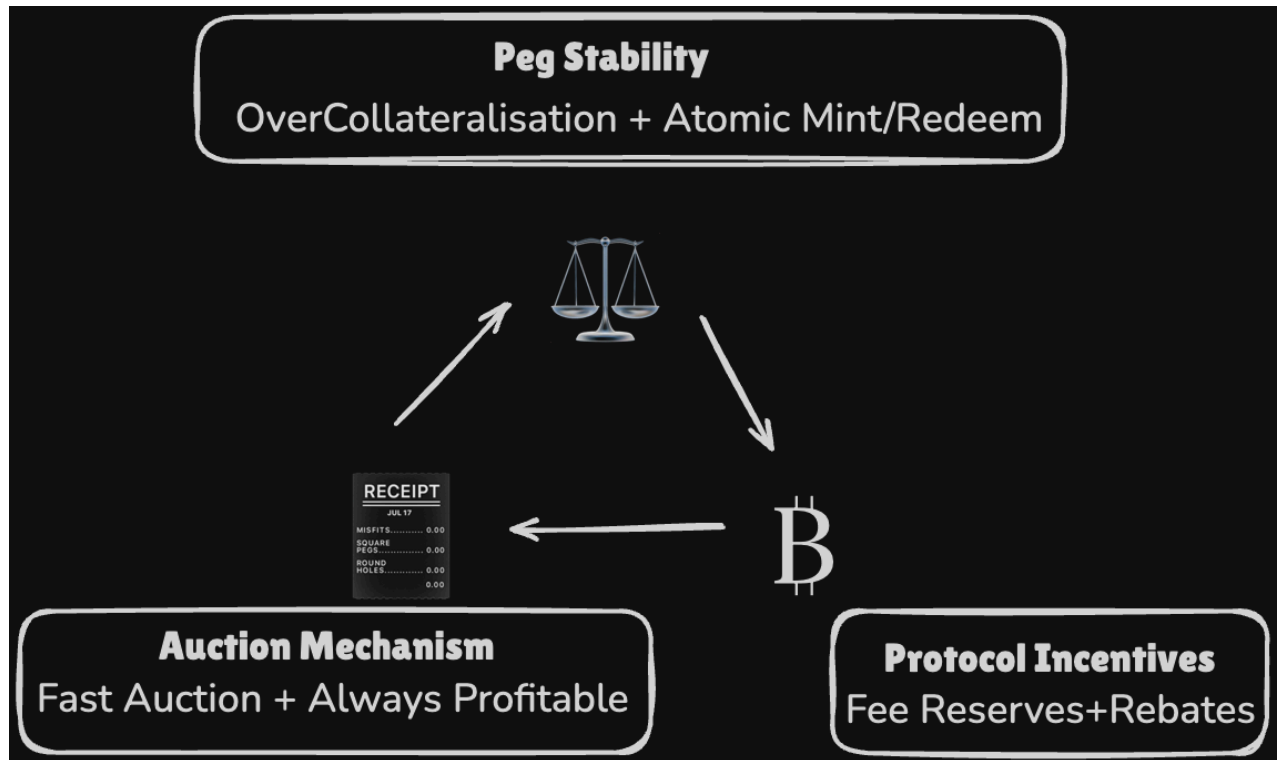
Each path is protected by:

- Transparent Bitcoin Script
- IC threshold Schnorr signatures
- On-chain validation
- No custodial entities

Everything is verifiable, inspectable, and censorship-resistant.

Stability

Ensuring resilience under all market conditions.



bUSD's peg relies on two pillars.

First, overcollateralisation ensures the value of locked Bitcoin always exceeds outstanding bUSD tokens by a configurable margin (e.g., 130%). With ICP, this becomes even stronger: collateral is validated using XRC's institutional-grade BTC/USD oracle, and enforcement occurs through threshold Schnorr-secured Taproot vaults derived directly from the Internet Computer's decentralized key infrastructure.

Second, atomic on-chain minting and redemption—spanning Bitcoin and the Internet Computer—enable arbitrageurs to correct price deviations instantly. When bUSD trades at a premium, users mint via an ICP-verified PSBT and sell; when it trades at a discount, traders redeem and buy back, restoring parity without external market makers. ICP's low-latency, deterministic execution ensures these mint/redeem paths always operate reliably and without custodians.

Auction-Backed Liquidations

To address undercollateralised positions, bUSD uses a continuously running, fee-aware auction that adapts to Bitcoin transaction fee dynamics.

Powered by ICP's autonomous timers and threshold Schnorr signatures, the auction ensures collateral is liquidated as soon as it becomes unsafe.

The auction algorithm maintains fairness and profitability by:

- validating price inputs using XRC
- authorizing liquidation paths via ICP threshold signatures
- executing state transitions atomically inside the bUSD canister

Every completed auction yields protocol fees—turning liquidations into revenue rather than cost. Even during Bitcoin network congestion, ICP's execution layer guarantees reliable, profitable liquidations.

Protocol Incentives

USDB aligns participant incentives to bolster stability:

Fee Reserves

A share of minting, redemption, and auction fees accrues to a reserve pool.

On ICP, this reserve is secured via decentralized threshold key control and used during extreme market conditions to offset elevated Bitcoin fees—providing a resilience buffer for system operations.

Auction Rebates

Bidders receive fee discounts or rebates, encouraging deep, responsive liquidity in liquidation events.

Because ICP enables predictable scheduling and signature automation, bidders operate with confidence, improving auction participation and clearing efficiency.

System Architecture

This page describes the complete end-to-end architecture of bUSD, powered by the Internet Computer (ICP) and Bitcoin's native infrastructure.

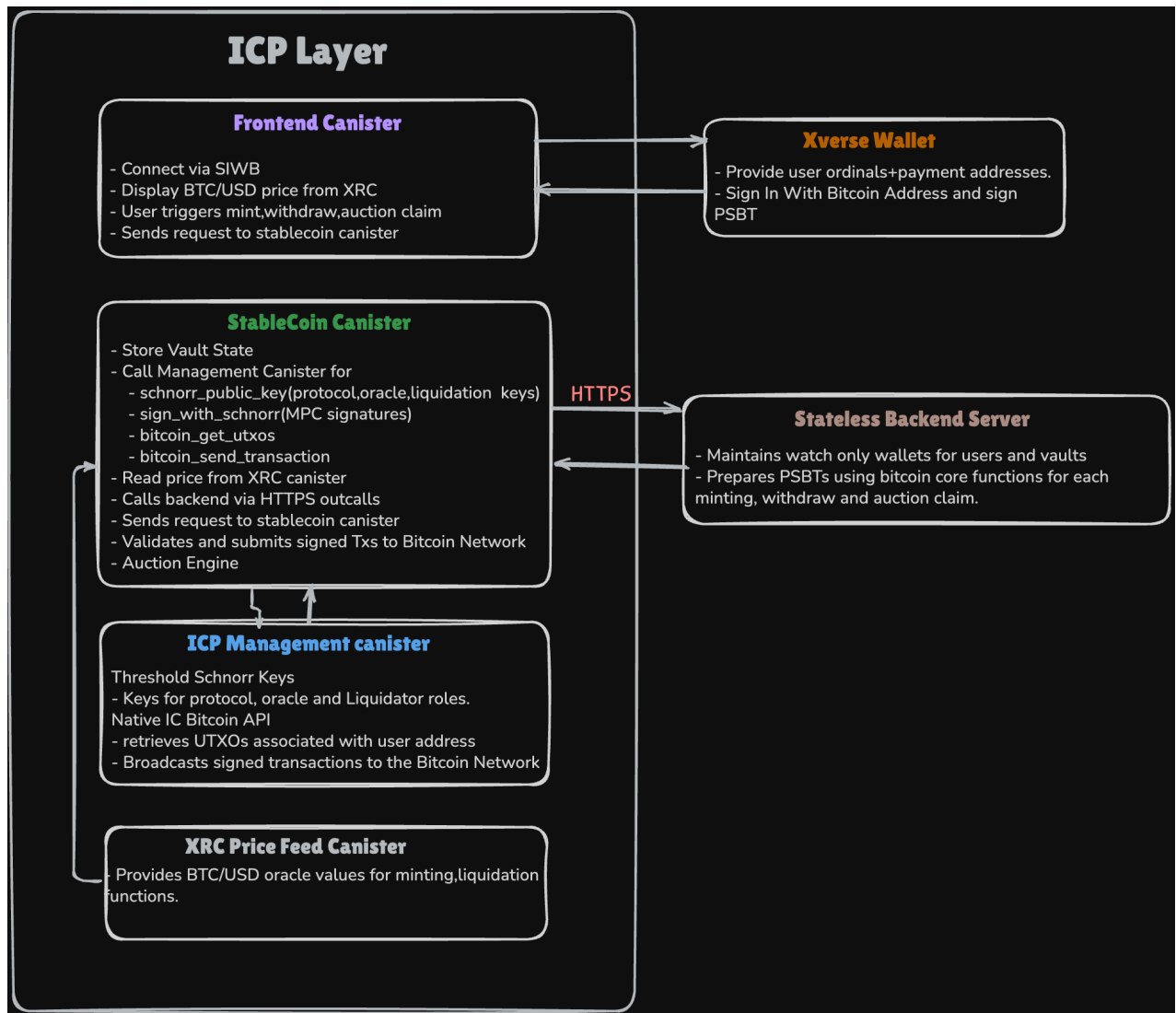
High-Level Design Overview

bUSDZ is built using:

- Stablecoin Canister — core business logic
- Frontend Canister — React/Vite UI hosted on IC
- XRC Canister — BTC/USD oracle feed
- ICP Management Canister — threshold Schnorr keys + native Bitcoin API
- Backend Stateless Server — PSBT construction using Bitcoin Core
- Xverse Wallet — SIWB + PSBT signing

Together, these components allow Bitcoin-backed stablecoin operations to run fully on-chain without custodians.

Architecture Diagram



Component Breakdown

1. Frontend Canister

- Serves React/Vite app from ICP
- Connects via SIWB
- Displays BTC/USD from XRC
- Sends mint/withdraw/auction requests to Stablecoin Canister

2. Xverse Wallet

- Provides user ordinals + payment addresses
- Signs PSBTs/messages
- Fully self-custodial

3. **Stablecoin Canister(Core Logic)**

- Stores vault state
- Derives protocol/oracle/liquidator MPC keys via schnorr_public_key
- Signs Bitcoin transactions via sign_with_schnorr
- Fetches price using XRC
- Makes HTTPS outcalls to backend to construct PSBTs
- Validates PSBT signatures
- Submits transactions to Bitcoin mainnet/testnet
- Runs liquidation auctions

4. **Backend Stateless Server**

- Maintains watch-only wallets
- Constructs PSBTs using Bitcoin Core
- Protected via x-api-key
- No private key material stored

5. **ICP Management Canister**

- Hosts threshold Schnorr keys for all protocol roles
- Provides Bitcoin Native API:
 - bitcoin_get_utxos
 - bitcoin_send_transaction
 - bitcoin_get_current_fee_percentiles
- Fully trustless + replicated

6. **XRC Price Feed Canister**

- Returns BTC/USD price
- Used for collateral calculation + liquidation thresholds

System Interaction Flow

Minting

1. User connects with Xverse
 2. Frontend requests mint → Stablecoin canister fetches price
 3. Stablecoin canister derives keys and calls backend
 4. Backend builds PSBT
 5. User signs PSBT in Xverse
 6. Canister validates + broadcasts via Bitcoin API
-

Redemption/Withdraw

1. User requests redeem
 2. Backend builds burn+unlock PSBT
 3. User signs
 4. Stablecoin canister finalizes + broadcasts
 5. Collateral unlocked through Bitcoin Script
-

Liquidation Auction

1. Ratio <112% → auction triggered
2. Stablecoin canister derives oracle & liquidation MPC keys
3. Auction PSBT prepared
4. Winning bidder claims collateral
5. Protocol earns fees

Why a Bitcoin-Native Stablecoin

The world's most secure asset deserves its own stablecoin — not a wrapped version, not a bridged derivative, not an IOU printed by a custodian.

1. Bitcoin is the deepest, most censorship-resistant collateral

Stablecoins today rely on banks, custodians, or opaque corporate structures.

Bitcoin offers:

- Globally verifiable collateral
- Immutable settlement
- No dependency on regulated banks
- Fully auditable vault scripts

A stablecoin built *directly* on Bitcoin inherits the strongest security model in existence.

2. Bridges are the largest attack surface in crypto

Almost every major stablecoin hack involved:

- Wrapped Bitcoin
- Multisig custodians
- Cross-chain bridges

bUSD avoids all of this.

Every mint and burn is a Bitcoin transaction, fully transparent, enforceable by script, and validated by open indexers.

3. Perfect capital efficiency for Bitcoin holders

BTC holders want to:

- Borrow against BTC
- Earn yield
- Use BTC for payments
- Hedge volatility

But without selling or leaving their wallet.

A Bitcoin-native stablecoin unlocks all of this — without compromising self-custody.

4. Stability without trust

bUSD relies on overcollateralization + atomic mint/burn + deterministic auctions.

No backing by banks.

No backing by treasuries.

No centralized issuer.

Just Bitcoin.

5. Unlocking the BTC economy

BTC is a \$1.2T asset with almost no native DeFi.

bUSD changes that by enabling:

- BTC-backed stable savings
- BTC-native payments
- BTC-native credit
- BTC-native automated auctions
- BTC-native arbitrage

Bitcoin becomes both the asset and the execution guarantee for a new monetary system.

Why ICP

bUSD is a Bitcoin-native stablecoin, but it depends on ICP because ICP provides the missing execution layer Bitcoin never had.

The Internet Computer brings smart-contract superpowers without compromising Bitcoin's security or sovereignty.

Why ICP complements Bitcoin perfectly

1. Native Bitcoin Integration (no bridges, no wrappers)

ICP is the only chain where smart contracts can call Bitcoin directly — full UTXO control, Schnorr signatures, and BTC transactions on mainnet BTC, not a bridge.

2. Threshold Schnorr Signatures (secure MPC built-in)

Our protocol requires protocol/oracle/liquidation signatures.

ICP gives us threshold Schnorr signing through the management canister — meaning:

- No custom MPC infrastructure
- No external custodians
- Keys never exist in one place
- IC's replicated threshold ECDSA/Schnorr service secures every vault

This is institutional-grade MPC, available out of the box.

3. HTTPS Outcalls for deterministic backend communication

ICP's boundary nodes securely hash and sign remote responses, letting the stablecoin canister talk to our backend with deterministic guarantees.

No other chain offers this capability natively.

4. Ultra-cheap, predictable execution + cycle economy

Liquidation loops, price checks, confirmation polls, descriptor derivations, all require predictable compute.

ICP cycles give:

- Stable costs
- No gas spikes
- No MEV
- No base layer congestion issues

5. Web + backend + stablecoin logic all on one chain

Frontend is also deployed as a canister → full decentralization.

This is impossible on Bitcoin and impractical on EVM chains.

bUSD uses ICP not as an add-on — but as a foundational execution layer that brings life, logic, and automation to Bitcoin's raw collateral.

bUSD core operations

Mint

Create bUSD tokens by locking Bitcoin collateral in a single atomic transaction.

What does a valid Mint transaction contain

A valid USDBZ mint transaction must include the following Bitcoin outputs (in any order). These are enforced by the stablecoin canister, the XRC price feed, and the native Bitcoin API on ICP. All (except Change Output) must be present for the mint to be accepted.

- **OP_RETURN Output** : The OP_RETURN output carries the bUSD rune meta data—token ID, operation ("MINT"), amount, target output(rune recipient) index and other relevant metadata—in a single unspendable script.
- **Rune Recipient Output** : The Rune Recipient UTXO is the transaction output whose address receives and holds the newly minted bUSD runes, linking the OP_RETURN data to the actual token ownership.
- **Collateral Cell** : The Collateral Cell is the on-chain Bitcoin output that permanently holds the locked collateral, anchoring bUSD's value to BTC within the mint transaction. It ensures every minted rune is backed by verifiable Bitcoin locked in this script.
 - Collateral must always be above 130% of the minted bUSD value according to the current oracle price; a recommended collateral ratio of bUSD% helps protect against price volatility and oracle delays, preventing liquidation.
 - The collateral amount is sent to a Taproot address protected by the following script.

```
OP_IF
  <protocol_publicKey>
  <user_publicKey>
OP_ELSE
  <liquidation_agent_publicKey>
  <oracle_publicKey>
OP_ENDIF
```

- **Fee Output** : The fee output is the amount paid by the user to cover protocol fees during the minting process. (For demo this is fixed at 1000sats per mint)

- ***Change Output:*** The change output is optional and is used to return any excess Bitcoin collateral back to the user after minting, depending on the transaction specifics.

How to Mint USDB tokens

sequenceDiagram

```

participant U as User
participant X as Xverse Wallet
participant FE as Frontend Canister
participant SC as Stablecoin Canister
participant XRC as XRC Oracle
participant MGMT as IC Mgmt (Schnorr + BTC API)
participant BE as Backend (PSBT Builder)
participant BTC as Bitcoin Network

U->>X: Connect (SIWB)
X-->>FE: payment + ordinals addresses

FE->>SC: mint_request(amount, addresses)

SC->>XRC: get_price()
XRC-->>SC: price

SC->>MGMT: derive_schnorr_keys(key_1)
MGMT-->>SC: protocol/oracle/liquidation x-only keys

SC->>MGMT: get_utxos(payment_address)
MGMT-->>SC: utxo_set

SC->>BE: HTTPS build_psbts(payload + utxos)
BE-->>SC: psbt_base64

SC-->>FE: psbt_base64
FE->>X: signPsbt(psbt_base64)
X-->>FE: signed_psbts

FE->>SC: finalize_mint(signed_psbts)

SC->>MGMT: bitcoin_finalize_psbts → tx_hex
MGMT-->>SC: tx_hex

SC->>MGMT: bitcoin_broadcast(tx_hex)
MGMT->>BTC: broadcast

loop until confirmations >= required
    SC->>MGMT: get_tx_status(txid)
    MGMT-->>SC: confirmations
end

SC-->>U: vault active + bUSDZ minted

```

1. Calculate Required Collateral

After User clicks on Mint button on frontend, Stablecoin canister fetches BTC/USD from XRC and computes minimum collateral (130%).

bUSD requires a 130% collateral ratio because it provides a mathematically safe buffer against Bitcoin's short-horizon volatility. Historical minute-level BTC/USD data shows that the 95th-percentile 30-minute drawdowns cluster around ~8–10%. When we add real-world constraints—wallet signing latency, network congestion, slow keeper response, and fee spikes—the safe operating range quickly expands. A 130% ratio guarantees that even fast crashes cannot push a freshly minted vault into danger before the system can react.

The additional overhead above the liquidation threshold creates the space needed for the 30-minute auction period. During this window, BTC can continue to drift downward by another ~5–7% on average (95th-percentile forward drift). The 130% ratio absorbs both the initial shock and the drift while still leaving surplus collateral. This ensures that every minted USDBZ token is always safely overcollateralized, maintaining solvency even during volatile market conditions.

2. Backend Builds PSBT (via HTTPS Outcall)

- stablecoin canister derives protocol/oracle/liquidation keys via threshold Schnorr
- canister retrieves all UTXOs under user's payment address using ICP Native Bitcoin API.
- canister prepares payload
- HTTPS outcall is sent to backend
- backend constructs PSBT (inputs + OP_RETURN + vault output + fee output)

3. User Signs PSBT in Xverse

Frontend Canister sends PSBT to user xverse wallet → user signs with their own keys.

4. Stablecoin Canister Finalizes

Canister validates

- Canister validates and finalizes the transaction
- Broadcasts it to Bitcoin network through Native ICP Bitcoin API.
- Records the created vault details in the vault database on the canister.
- Once confirmed on-chain, the vault becomes active and bUSD becomes spendable.

Atomic minting ensures every USDBZ is fully backed at birth.

Burn

Burn USDB tokens and get back locked collateral in a single atomic transaction.

What does a valid Burn transaction contains

-  The following UTXOs can be present in any order in the transaction, but all must be present for the transaction to be considered valid by USDB protocol rules.

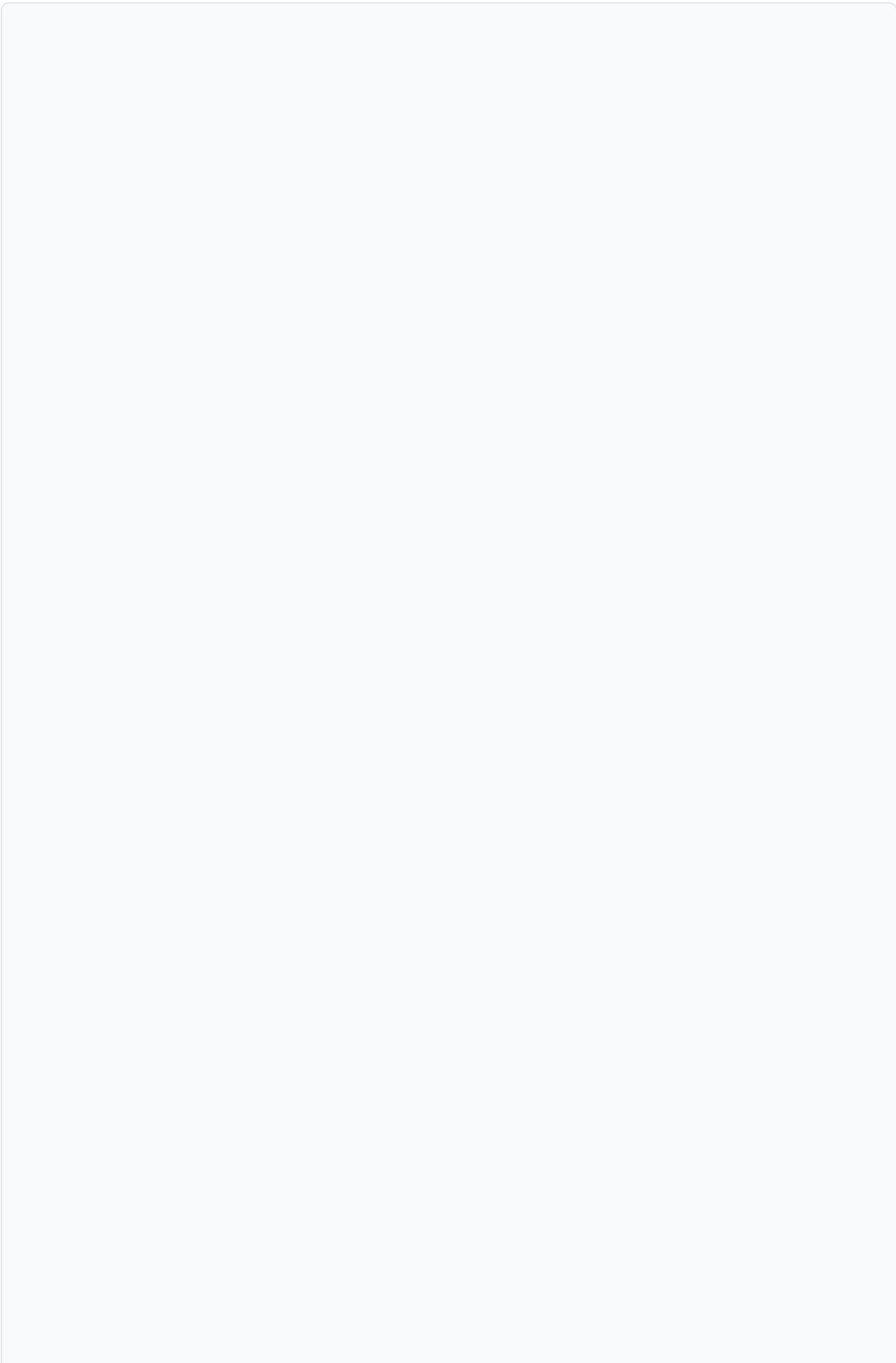
Inputs

- **USDB token recipient UTXO/UTXOs** : The transaction spends one or more UTXOs containing bUSD runes that the user wishes to burn, referencing the specific token amounts to be destroyed for collateral release.
- **Collateral Cell** : The UTXO which holds the locked collateral.

Outputs

- **OP_RETURN Output** : The OP_RETURN output carries the bUSD rune data—token ID, operation ("BURN"), amount and other relevant metadata—in a single unspendable script.
- **Collateral(*+change*) Release Output** : The collateral locked with minting is released to user (*along with any change amount after funding this transaction*).

How to Burn bUSD tokens



sequenceDiagram

```

    participant U as User
    participant X as Xverse Wallet
    participant FE as Frontend Canister
    participant SC as Stablecoin Canister
    participant MGMT as IC Mgmt<br/>(Schnorr + BTC API)
    participant BE as Backend<br/>(PSBT + finalize)
    participant BTC as Bitcoin Network
  
```

```

    U->>X: Connect (SIWB)
  
```

```

    X-->>FE: auth + addrs
  
```

```

    U->>FE: request burn(vault_id, amount)
  
```

```

    FE->>SC: burn_request(vault_id, amount, payout_address)
  
```

```

    %% UTXO discovery + PSBT build
  
```

```

    SC->>MGMT: get_utxos(rune + collateral)
  
```

```

    MGMT-->>SC: rune_utxos + collateral_cell
  
```

```

    SC->>BE: HTTPS build_burn_psbt(payload, utxos)
  
```

```

    BE-->>SC: unsigned_psbt
  
```

```

    %% user signs
  
```

```

    SC-->>FE: unsigned_psbt
  
```

```

    FE->>X: signPsbt(unsigned_psbt)
  
```

```

    X-->>FE: user_signed_psbt
  
```

```

    FE->>SC: submit_signed_psbt(user_signed_psbt)
  
```

```

    %% protocol signature from IC
  
```

```

    SC->>MGMT: sign_with_schnorr(key_1, vault_path, sighash)
  
```

```

    MGMT-->>SC: protocol_signature
  
```

```

    %% send to backend for merge + finalize
  
```

```

    SC->>BE: HTTPS finalize_burn_psbt(user_signed_psbt,
    protocol_signature)
  
```

```

    BE->>BE: merge sigs + finalize tx
  
```

```

    BE-->>SC: tx_hex
  
```

```

    %% broadcast from canister via Bitcoin API
  
```

```

    SC->>MGMT: bitcoin_broadcast(tx_hex)
  
```

```

    MGMT->>BTC: broadcast
  
```

```

    loop until confirmations >= required
  
```

```

        SC->>MGMT: get_tx_status(txid)
  
```

```

        MGMT-->>SC: confirmations
  
```

```

    end
  
```

```

    SC-->>U: burn complete → collateral released
  
```

1. Burn Request Submitted

User selects vault → frontend calls stablecoin canister burn_request.

The stablecoin canister queries its vault database to fetch:

- rune UTXOs
- vault collateral cell

3. PSBT Construction

The canister makes an HTTPS outcall to the backend to build the burn PSBT (unsigned).

4. User Signature

The PSBT is returned → Xverse signs user-owned inputs and the vault withdraw path signature (other is protocol's).

5. Protocol Signature

The frontend sends user-signed PSBT back to the canister.

The canister calls:

```
sign_with_schnorr {  
  key_id = key_1,  
  message = sighash,  
  derivation_path = vault_derived_path  
}
```

This generates the protocol leaf signature required to spend the collateral cell.

6. Finalize + Broadcast

The Stablecoin Canister sends both:

- user_signed_psb
- protocol_signature

to the backend via HTTPS outcall.

Backend responsibilities:

- Insert both signatures into the PSBT
- Validate script path + enforce protocol rules
- Run:
 - finalizepsbt
 - extractorawtransaction
- Produce final tx_hex and sends back to the canister.
- IC Bitcoin API broadcasts the final transaction to testnet4/mainnet.

Making the burn atomic ensures every USDB token that is removed from circulation corresponds to the unlocking of the appropriate amount of Bitcoin collateral, preserving system solvency and maintaining trustworthiness.

Auction

When a user's collateral health drops below the protocol's safety threshold, their locked BTC is automatically put up for auction to maintain solvency and ensure every bUSD token remains fully and safely backed.

When is a user's collateral put up for auction?

A user's collateral enters auction when the collateral ratio falls below the Threshold Collateral Ratio (TL).

$$\text{Collateral Health} = \frac{(\text{Deposited BTC} \times \text{Current USD/BTC Price})}{\text{Lot/Mint Amount (USD)}} \times 100$$

Condition:

$$\text{Collateral Health} < \text{TL}$$

If the above condition is true, then the cell is considered undercollateralized and enters the liquidation auction.

Example:

TL = 112% and the user's collateral health drops to 110%, their vault enters auction.

The liquidation threshold is set at 112% because this level marks the boundary where vaults begin to approach statistically unsafe conditions. Our analysis shows that once collateral falls below ~112%, even a typical 30-minute BTC dip could render the vault undercollateralized. Triggering auctions at 112% ensures the system reacts early, before risk compounds, and while enough collateral remains to run a profitable descending-price auction.

By starting auctions at 112%, the protocol leaves roughly an 18% buffer (between 130% and 112%) to safely unwind collateral during the auction window. This buffer absorbs ongoing price drift, covers rising fees, and prevents liquidation losses. The descending-price mechanism then gradually frees more collateral over time, guaranteeing a bidder will take the position while the vault is still solvent. This maintains system integrity and ensures liquidations remain profitable rather than costly.

On ICP, this liquidation logic is enforced by the Stablecoin canister, which runs a periodic timer and talks to on-chain services:

- 60-second health check loop
 - A timer inside the Stablecoin canister runs roughly every 60 seconds.
 - It scans the on-chain vault database for all open positions.
 - For each vault, it fetches the latest BTC/USD price from the XRC price feed canister and recomputes the collateral health.
- Triggering auction state
 - If Collateral Health < TL, the Stablecoin canister marks that vault as "in auction" and schedules it for liquidation.

Auction Algorithm

- **Auction Duration:** 30 minutes
- **Start Trigger:** Collateral ratio falls below TL
- **Purpose:** Gradually offer increasing portions of the user's locked BTC for purchase using burned USDB runes.

- **Participants:** Anyone can buy the collateral by burning *10(or whatever is the mint amount)* USDB runes (per cell) and paying network fees
- **Unclaimed Collateral:** Any remaining BTC after the auction closes is transferred to the Protocol Treasury to maintain system balance and cover bad debt.

The auction process follows a deterministic and transparent formula, ensuring that all participants know exactly how much collateral will be offered at any given timestamp. This predictability creates a level playing field for all arbitrageurs and encourages fast, competitive liquidation.

At every minute t ($0 \leq t \leq 30$), the amount of collateral available for purchase increases linearly according to the following formula:

$$\text{Collateral Offered}(t) = \left[\frac{((TL - 100)/30 \times t + 100)}{TL} \right] \times \text{Locked Collateral}$$

Example with TL = 112% :

t = 0	$(100/112) \times \text{Collateral} \approx 89.29\%$	Auction starts — minimal collateral offered
t = 15	$(106/112) \times \text{Collateral} \approx 94.6\%$	Mid-auction — more BTC available
t = 30	$(112/112) \times \text{Collateral} = 100\%$	Auction ends — full collateral offered

flowchart LR

```

    A[Vault health check\n(canister timer or force_start)] -->|ratio < 112%| B[Start auction\nstore AuctionState]
    B --> C[Frontend lists auctions\n(list_auctions)]
    C --> D[User clicks Claim]
    D --> E[Frontend supplies ordinals & payment info\n+ current claim price]
    E --> F[Canister.prepare_auction_claim\nHTTP -> backend /auction/prepare]
    F --> G[Backend builds PSBT\nreturns PSBT/metadata]
    G --> H[Frontend shows PSBT\nuser confirms]
    H --> I[User signs ordinals input via Xverse]
    I --> J[Frontend sends signed PSBT\n→ canister.finalize_auction_claim]
    J --> K{Backend /auction/finalize response}
    K -->|needs oracle/liquidation sigs (202)| L[Canister runs oracle check\nsign_with_schnorr for oracle & liquidation roles]
    L --> M[Backend finalize again\nwith signatures]
    M -->|already complete| K
    M --> N[Canister broadcasts via bitcoin_send_transaction]
    N --> O[Canister marks auction claimed\nupdates vault store]
    O --> P[Frontend refresh\nlist_user_vaults shows success]
  
```

Economic Intuition

- At $t = 0$, approximately 89.29% of the locked collateral is offered. Buying it with 10 USDB represents a zero-arbitrage opportunity (ignoring network fees).
- As time progresses, the amount of BTC available for the same 10 USDB gradually increases, creating a growing arbitrage incentive that encourages faster liquidation.
- The auction curve is fully predictable, giving all arbitrageurs equal information about how collateral availability changes over time. This shared visibility creates a common ground for competition, pushing for the fastest and most efficient liquidation possible.
- This mechanism ensures the protocol remains solvent, minimizes undercollateralized exposure, and aligns all participants toward maintaining system stability.

Arbitrage Condition

An arbitrage opportunity exists when:

$$\text{Collateral Offered (BTC)} \times \text{Current USD/BTC Price} > \text{Lot Amount (USD)}$$

At this point, arbitrageurs can profitably purchase the collateral by burning *10(or whatever is the mint amount)* USDB runes.

SIWB

SIWB securely links the user's Xverse wallet to the protocol, proving identity and pre-warming all Bitcoin watch-only wallets.

SIWB (Sign-In With Bitcoin) via Xverse is the authentication and initialization layer that makes the entire bUSDZ user experience smooth, secure, and *Bitcoin-native*.

It ensures the protocol knows who the user is, which Bitcoin keys they control, and prepares all backend Bitcoin-Core infrastructure ahead of time.

Why SIWB matters

Without SIWB:

- The backend has no idea which payment or ordinals addresses belong to the user.
- Watch-only wallets would have to be created/imported on-demand, causing 45+ seconds of delay for each core operation and thereby https calls fail/need multiple tries.
- Descriptor imports might trigger wallet rescans, slowing down minting/withdraw.
- The front end cannot authenticate requests to the backend reliably.

SIWB solves all of this with a single user action: Connect Xverse.

What Happens When the User Connects Xverse

1. User Signs a SIWB Challenge

- The frontend requests a SIWB signature using Xverse.
- This proves cryptographically that the user controls:
 - ✓ their payment address
 - ✓ their ordinals address

This signature removes all centralized authentication — identity = your Bitcoin keys.

2. Frontend Sends the Signed Payload to Backend

- The backend validates the signature.
 - Issues a short-lived session token that the frontend uses for mint/withdraw/auction requests.
 - No passwords, no OAuth, no centralized identity.
-

3. Backend Automatically Performs Wallet Warm-Up

Immediately after SIWB validation, the backend does all Bitcoin-Core prep work:

1. Creates/loads 3 watch-only wallets
 - a. payment-wpkh (Tracks user's spendable BTC)
 - b. ordinals-tr (Tracks user's spendable Runes)
2. Imports descriptors (with timestamp = 0)
 - a. This ensures:
 - No rescan delays
 - walletcreatefundedpsbt works instantly
 - walletprocesspsbt runs without fetching chain history
3. Caches descriptors + internal state

Future PSBT builds (mint, withdraw, auction) become instant.

```
sequenceDiagram
    participant U as User
    participant X as Xverse Wallet
    participant FE as Frontend Canister
    participant BE as Backend (Node)
    participant BC as Bitcoin Core

    U->>X: Click "Connect Xverse"
    X->>FE: Expose payment + ordinals pubkeys
    FE->>X: Request SIWB signature (challenge)
    X-->>FE: SIWB signed message

    FE->>BE: POST /siwb { pubkeys, signature }
    BE->>BE: Verify SIWB signature
    BE-->>FE: Issue session token

    note over BE: Wallet warm-up starts

    BE->>BC: create/load watch-only payment wallet (wpkh)
    BE->>BC: create/load watch-only ordinals wallet (tr)
    BE->>BC: create/load vault wallets for existing vaults

    BE->>BC: importdescriptors (timestamp = 0)
    BC-->>BE: descriptors imported (no rescan delay)

    note over FE,BE: Session ready for mint/withdraw/auction<br/>with pre-warmed wallets & descriptors
```

ICP's canisters cannot run Bitcoin Core — so the backend must.

But to maintain performance, the backend must already know:

- which wallets belong to the user
- which descriptors to load
- which vaults exist for them

SIWB provides exactly this:

Stateless canister, stateful Bitcoin backend, perfectly synchronized via SIWB.

Zero delays, Bitcoin-native UX, and cryptographic authentication.

Security & Speed

ICP's canisters cannot run Bitcoin Core — so the backend must.

But to maintain performance, the backend must already know:

- which wallets belong to the user
- which descriptors to load
- which vaults exist for them

SIWB is not “just auth” — it is the glue that makes the entire protocol work smoothly:

- Bitcoin-native user identity
- Instant PSBT construction
- Stateless canister, stateful backend
- Zero rescan latency
- Perfect UX for mint/withdraw/auction

It is the reason the protocol feels fast, secure, and purely Bitcoin-first, even though it runs on ICP.